Samuel et Liam

IDIA3 - AL (p4)

→ On a au moins 3 points avec des abscisses différentes, donc les colonnes de X sont linéairement indépendantes. La dimension de l'espace engendré par les colonnes de X (Col(X)) est donc de 3.
Les deux dernières colonnes (celles avec les x_i et les 1) étaient exactement celles utilisées précédemment pour l'ajustement par une ligne droite.

③ La nouvelle colonne (contenant les x_i^2) n'est pas une combinaison linéaire des 2 dernières colonnes (celles avec les x_i et les 1).
La nouvelle colonne ajoute une dimension supplémentaire à l'espace Col(X), donc cet espace est plus grand. Donc, la projection $\hat{y}$ sur ce nouvel espace sera toujours plus proche (ou au pire, à la même distance) du vecteur cible y, qu'elle ne l'était sur l'ancien plan.

④ (Voir code exo 9 Annexe 5)

⑤ On a déjà x et y qui donnent une dimension 2 donc pour la dimension 3, il nous manque que l'axe z tel que $z = ax + by + c$ et on connait déjà les valeurs de a, b et c.
Il nous reste plus qu'à appliquer la formule des moindres carrés comme précédemment.
(Voir le code sur notebook exo 9)

ⓓ <u>Evaluation de la qualité du modèle</u>

① Voir Annexe 5

② Pour que chaque donnée du modèle serve exactement 3 fois, on fait une validation croisée à 4 blocs. On commence

donc par diviser les 20 points (des fichiers x et y .csv) en 4 blocs de 5 points. À chaque tour, on garde 1 bloc pour le test (25%) et les 3 autres pour l'entraînement (75%). On calcule ensuite l'erreur moyenne sur les 4 tours pour obtenir une estimation. Enfin, on répète les étapes pour les polynomes de degrés 1, 2, 3... pour trouver le meilleur.
(voir code annexe 5).
Ici, le meilleure modèle est celui avec un polynome de degrés 4, car il à le taux d'erreur le plus faible.

10- Application linéaire pour les codes correcteur binaires

① On définit $v = (0110)$
On fait $v \cdot G$ en vérifiant le modulo 2 :

$$mot = v \times G = (0110)\begin{pmatrix} 1000110 \\ 0100101 \\ 0010011 \\ 0001111 \end{pmatrix} = \begin{pmatrix} 0\times1 + 1\times0 + 1\times0 + 0\times0 \\ 0\times0 + 1\times1 + 0 + 0 \\ 0 + 0 + 1 + 0 \\ 0 + 0 + 0 + 0 \\ 0 + 1 + 0 + 0 \\ 0 + 0 + 1 + 0 \\ 0 + 1 + 1 + 0 \end{pmatrix} = \begin{pmatrix} 0 \\ 1 \\ 1 \\ 0 \\ 1 \\ 1 \\ 0 \end{pmatrix}$$

$\hookrightarrow \equiv 0 [2]$

② On définit d'abord :

$$H = [P_{4,3} | I_{3,3}]^T = \left(\begin{array}{cccc|ccc} 1 & 1 & 0 & 1 & 1 & 0 & 0 \\ 1 & 0 & 1 & 1 & 0 & 1 & 0 \\ 0 & 1 & 1 & 1 & 0 & 0 & 1 \end{array}\right)^T = \begin{pmatrix} 1 & 1 & 0 \\ 1 & 0 & 1 \\ 0 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

Puis on fait la vérif :

$$v \cdot G \cdot H = mot \cdot H = (0110110) \cdot \text{''} = (1+1 \quad 1+1 \quad 1+1) = (0\,0\,0)$$

③ $mot_faux = (0\,\underline{0}\,1\,0\,1\,1\,0)$
donc on obtient $mot_faux \cdot H = (1 \quad 1+1 \quad 1) = (1\,0\,1) \neq (0\,0\,0)$
Une anomalie est bien détectée.
Pour trouver l'erreur sans tout recommencer, on peut retracer la coquille par les 1 du produit final pour en déduire notre bit erroné.

Samuel
Liam

IDIA
AL
P3

④ double_faute = (0 0 0 0 1 1 0)
double_faute · H = (1 1 0) qui est différend du résultat de la question 3 donc on ne pourra pas retrouver le bit erroné qui est commun à mot_faux.
On en déduit donc que l'on peut tracer les erreurs que lorsqu'il y a une seule. Ce dont on ne peut pas être sûr.

⑤ triple_faute = (0 0 0 0 0 1 0)
triple_faute · H = (0 1 0) ≠ (0 0 0)
triple_faute_v2 = (1 0 0 0 1 1 0)
triple_f_v2 · H = (1+1 1+1 0) = (0 0 0)
On retombe sur le vecteur nul alors que l'on sait que notre mot est faux.
On en déduit que la méthode de vérification actuelle n'est pas exploitable tant que ~~l'on sait pas~~ plusieurs erreurs sont possibles.

⑥ méthode calculatoire :

$$G \cdot H = \begin{pmatrix} 1 & 0 & 0 & 0 & 1 & 1 & 0 \\ 0 & 1 & 0 & 0 & 1 & 0 & 1 \\ & & \cdots & & & & \end{pmatrix} \cdot \begin{pmatrix} 1 & 1 & 0 \\ 1 & 0 & 1 \\ 0 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 0 & 1 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix} = \begin{pmatrix} 1+1 & 1+1 & 0 \\ 1+1 & 0 & 1+1 \\ & \cdots & \end{pmatrix} = 0_7$$

méthode théorique (par blocs) :
$G = (I_4 | P_{4,3})$ et $H = (P_{4,3} | I_3)^T$ donc on a
$G \cdot H = (I_4 | P) \cdot \begin{pmatrix} P \\ I_3 \end{pmatrix} = I_4 \times P + P \times I_3 = P + P \equiv 0 \; [2]$
Ainsi, dans le calcul de vérif° $v \cdot G \cdot H$, on obtient
⑦ $v \cdot (G \cdot H) = v \cdot 0 = (0 \; 0 \; 0)$ donc le mot est ok

⑧ Le vecteur $v \cdot G \in E$ ~~donc~~ avec $\dim(E) = 7$ car le résultat du produit est un vecteur de 7 bits.
De plus, ces mots obtenus sont des combinaisons linéaires des 4 lignes de G qui sont linéairement indépendantes. Elles créent donc un sous espace vectoriel de dimension 4.
Non car $2^4 < 2^7$.

⑨ "Distance de Hamming : 2 mots différents auront forcément 3 bits différent."
Pour nos 3 tests nous avons eu des écarts de 3, 4 et 3, ce qui valide notre loi ci-dessus.

8 - Espace de fonctions

① Dans $\mathbb{R}^n$, le produit scalaire de 2 vecteurs (par exemple u et v) est la somme des produits, composante par composante ($\sum_{i=1}^{n} u_i v_i$). Donc on a bien une interprétation proche. Si on discrétise l'intervalle [a, b] en plusieurs points x_i, séparés par un écart Δx, on peut faire une approximation de l'intégrale comme ceci : $\sum f(x_i) g(x_i) \Delta x$. Les valeurs de la fonction pour tous les x_i agissent comme les coordonnées d'un grand vecteur. Cette intégrale est donc une version "continue" du produit scalaire classique.

② On a déjà un produit scalaire valide, donc on peut transposer toutes les fonctions géométrique aux fonctions. On a donc : - Deux fonctions f et g sont orthogonales si $\langle f, g \rangle = 0$, c'est à dire si $\int_a^b f(x)\, g(x)\, dx = 0$.
- Norme euclidienne : $\|f\| = \sqrt{\langle f, f \rangle} = \sqrt{\int_a^b (f(x))^2 dx}$.
- L'angle θ est défini par $\cos(\theta) =$

③④ (voir code exo 8).

⑤ Grace aux questions précédentes, on peut formuler cette hypothèse : La famille de fonctions composées des sinus, cosinus et de fréquences entières multiples, forme une famille orthogonale sur l'intervalle $[-\pi, \pi]$ (c'est la base de Fourrier).

⑥ (voir code exo 8 # Question 6). *

⑦ Si on fait le produit scalaire $\langle f_i, \sin(nx) \rangle$, grace à la linéarité du produit scalaire et son orthogonalité, tous les termes s'annulent, sauf, celui correspondant exactement à $\sin(nx)$.
Cela peut être utile pour "extraire" des coefficients. En effet, ca permet de déterminer l'amplitude d'une fréquence $\sin(nx)$ contenue dans un signal f par exemple.

* (suite ⑥) : On observe que les fonctions semble converger vers la fonction $\sin(x)$ (visuelement ∩).